

Lab Work

Program 1:

Code:

```
1  class BankAccount:
2
3      def __init__(self, account_number, account_holder, initial_balance=0.0):
4          self.account_number = account_number
5          self.account_holder = account_holder
6          self.balance = initial_balance
7
8      def deposit(self, amount):
9          if amount <= 0:
10             print("Deposit amount must be positive.")
11             return
12          else:
13             self.balance += amount
14             return self.balance
15
16      def withdraw(self, amount):
17          if amount <= 0:
18             return print("Withdrawal amount must be positive.")
19          elif amount > self.balance:
20             return print("Insufficient funds!")
21          else:
22             self.balance -= amount
23             return self.balance
24
25      def display(self):
26          print(f"Account Holder: {self.account_holder}")
27          print(f"Account Number: {self.account_number}")
28          print(f"Current Balance: ${self.balance}")
29          return self.balance
30
31  if __name__ == "__main__":
32      account = BankAccount("019", "Tayyab Imran", 1000.0)
33      #Deposit
34      account.deposit(500)
35      #Withdraw
36      account.withdraw(2000)
37      #Display
38      account.display()
```

Output:

```
Insufficient funds!  
Account Holder: Tayyab Imran  
Account Number: 019  
Current Balance: $1500.0
```

Program 2:

Code:

```
1  class Book:  
2      def __init__(self, title, author, isbn):  
3          self.title = title  
4          self.author = author  
5          self.isbn = isbn  
6          self.available = True  
7  
8      def __str__(self):  
9          status = "Available" if self.available else "Checked Out"  
10         return f"'{self.title}' by {self.author} (ISBN: {self.isbn}) - {status}"  
11  
12     def __eq__(self, other):  
13         if isinstance(other, Book):  
14             return self.isbn == other.isbn  
15         return False  
16  
17     def __hash__(self):  
18         return hash(self.isbn)  
19  
20  
21  class Library:  
22      def __init__(self):  
23          self.books = {}  
24  
25      def add_book(self, book):  
26          if not isinstance(book, Book):  
27              return print("Only Book instances can be added.")  
28          if book.isbn in self.books:  
29              return print(f"Book with ISBN {book.isbn} already exists in the library.")  
30          self.books[book.isbn] = book  
31          print(f"Added: {book}")  
32  
33      def search_by_title(self, title):  
34          if not title:  
35              return print("Search title cannot be empty.")  
36          title_lower = title.strip().lower()  
37          results = [book for book in self.books.values() if title_lower in book.title.lower()]  
38          return results
```

```
40 def search_by_author(self, author):
41     if not author:
42         return print("Search author cannot be empty.")
43     author_lower = author.strip().lower()
44     results = [book for book in self.books.values() if author_lower in book.author.lower()]
45     return results
46
47 def check_out(self, isbn):
48     if isbn not in self.books:
49         return print(f"Book with ISBN {isbn} not found in the library.")
50     book = self.books[isbn]
51     if not book.available:
52         return print(f"Book '{book.title}' is already checked out.")
53     book.available = False
54     print(f"Checked out: {book}")
55
56 def return_book(self, isbn):
57     if isbn not in self.books:
58         return print(f"Book with ISBN {isbn} not found in the library.")
59     book = self.books[isbn]
60     if book.available:
61         return print(f"Book '{book.title}' is already available (not checked out).")
62     book.available = True
63     print(f"Returned: {book}")
64
65 def list_available_books(self):
66     available = [book for book in self.books.values() if book.available]
67     if not available:
68         print("No books currently available.")
69     else:
70         print("Available books:")
71         for book in available:
72             print(f" - {book}")
73     return available
```

```

75     def list_all_books(self):
76         all_books = list(self.books.values())
77         if not all_books:
78             print("Library is empty.")
79         else:
80             print("All books in the library:")
81             for book in all_books:
82                 print(f" - {book}")
83         return all_books
84
85 if __name__ == "__main__":
86     library = Library()
87     book1 = Book("Mirza Ghalib", "Asadullah Khan", "101")
88     book2 = Book("1984", "George Orwell", "102")
89     book3 = Book("Mein Kampf", "Hitler", "103")
90
91     #Adding Books to the Library
92     library.add_book(book1)
93     library.add_book(book2)
94     library.add_book(book3)
95
96     #Searching for Book by Title
97     print("\nSearching for '198 Nerds':")
98     results = library.search_by_title("Mein")
99     for res in results:
100         print(res)
101
102     #Checking Out
103     library.check_out("101")
104
105     #Checking Available Books
106     library.list_available_books()
107
108     #Returning Book
109     library.return_book("103")
110     library.list_available_books()

```

Output:

```

Added: 'Mirza Ghalib' by Asadullah Khan (ISBN: 101) - Available
Added: '1984' by George Orwell (ISBN: 102) - Available
Added: 'Mein Kampf' by Hitler (ISBN: 103) - Available

Searching for '198 Nerds':
'Mein Kampf' by Hitler (ISBN: 103) - Available
Checked out: 'Mirza Ghalib' by Asadullah Khan (ISBN: 101) - Checked Out
Available books:
- '1984' by George Orwell (ISBN: 102) - Available
- 'Mein Kampf' by Hitler (ISBN: 103) - Available
Book 'Mein Kampf' is already available (not checked out).
Available books:
- '1984' by George Orwell (ISBN: 102) - Available
- 'Mein Kampf' by Hitler (ISBN: 103) - Available

```

Program 3:

Code:

```
1  class Student:
2      def __init__(self, student_id, name):
3          self.student_id = student_id
4          self.name = name
5          self.grades = []
6
7      def add_grade(self, grade):
8          self.grades.append(float(grade))
9          print(f"Added grade {grade} for {self.name}.")
10
11     def calculate_average(self):
12         if not self.grades:
13             print("No grades available to calculate average.")
14             return None
15         average = sum(self.grades) / len(self.grades)
16         return round(average, 2)
17
18     def get_letter_grade(self, average=None):
19         if average >= 90:
20             return 'A'
21         elif average >= 80:
22             return 'B'
23         elif average >= 70:
24             return 'C'
25         elif average >= 60:
26             return 'D'
27         else:
28             return 'F'
29
```

```

30     def display_info(self):
31         average = self.calculate_average()
32         letter = self.get_letter_grade(average)
33
34         print(f"Student ID: {self.student_id}")
35         print(f"Name: {self.name}")
36         print(f"Grades: {self.grades if self.grades else 'None'}")
37         if average is not None:
38             print(f"Average Grade: {average}")
39             print(f"Letter Grade: {letter}")
40         else:
41             print("Average Grade: N/A")
42             print("Letter Grade: N/A")
43
44     if __name__ == "__main__":
45         student = Student("019", "Tayyab Imran")
46         student.add_grade(85)
47         student.add_grade(90)
48         student.add_grade(78)
49         student.add_grade(92)
50
51         #Display Info
52         student.display_info()

```

Output:

```

Added grade 85 for Tayyab Imran.
Added grade 90 for Tayyab Imran.
Added grade 78 for Tayyab Imran.
Added grade 92 for Tayyab Imran.
Student ID: 019
Name: Tayyab Imran
Grades: [85.0, 90.0, 78.0, 92.0]
Average Grade: 86.25
Letter Grade: B

```

Program 4:

Code:

```
1  class MenuItem:
2      def __init__(self, name, price, category):
3          self.name = name
4          self.price = float(price)
5          self.category = category
6
7      def __str__(self):
8          return f"{self.name} ({self.category}): ${self.price:.2f}"
9
10
11 class Order:
12
13     def __init__(self, tax_rate=0.1):
14         self.items = []
15         self.tax_rate = float(tax_rate)
16         self.discount_percent = 0.0
17
18     def add_item(self, menu_item):
19         self.items.append(menu_item)
20         print(f"Added to order: {menu_item}")
21
22     def apply_discount(self, discount_percent):
23         self.discount_percent = float(discount_percent)
24         print(f"\nApplied {self.discount_percent}% discount.")
25
26     def get_subtotal(self):
27         subtotal = sum(item.price for item in self.items)
28         return round(subtotal, 2)
29
30     def get_discounted_subtotal(self):
31         subtotal = self.get_subtotal()
32         discount_factor = 1 - (self.discount_percent / 100)
33         discounted = subtotal * discount_factor
34         return round(discounted, 2)
35
36     def get_tax(self):
37         discounted_subtotal = self.get_discounted_subtotal()
38         tax = discounted_subtotal * self.tax_rate
39         return round(tax, 2)
```

```

41 def get_total(self):
42     discounted_subtotal = self.get_discounted_subtotal()
43     tax = self.get_tax()
44     total = discounted_subtotal + tax
45     return round(total, 2)
46
47 def generate_receipt(self):
48     if not self.items:
49         receipt = "Order is empty."
50         print(receipt)
51         return receipt
52
53     subtotal = self.get_subtotal()
54     discount_amount = subtotal - self.get_discounted_subtotal()
55     tax = self.get_tax()
56     total = self.get_total()
57
58     receipt_lines = ["\n----- Receipt -----"]
59     for item in self.items:
60         receipt_lines.append(str(item))
61     receipt_lines.append(f"\nSubtotal: ${subtotal:.2f}")
62     if self.discount_percent > 0:
63         receipt_lines.append(f"Discount ({self.discount_percent}%): -${discount_amount:.2f}")
64     receipt_lines.append(f"Tax ({self.tax_rate * 100}%): ${tax:.2f}")
65     receipt_lines.append(f"Total: ${total:.2f}")
66     receipt_lines.append("-----")
67
68     receipt = "\n".join(receipt_lines)
69     print(receipt)
70     return receipt

```

```

72 if __name__ == "__main__":
73     # Create menu items
74     item1 = MenuItem("Burger", 10.99, "Main Course")
75     item2 = MenuItem("Fries", 4.99, "Side")
76     item3 = MenuItem("Soda", 2.49, "Beverage")
77
78     # Create order with 8.5% tax
79     order = Order(tax_rate=0.085)
80
81     # Add items
82     order.add_item(item1)
83     order.add_item(item2)
84     order.add_item(item3)
85
86     # Apply 15% discount
87     order.apply_discount(15)
88
89     # Generate receipt
90     order.generate_receipt()

```


Output:

```
Added to order: Burger (Main Course): $10.99
Added to order: Fries (Side): $4.99
Added to order: Soda (Beverage): $2.49

Applied 15.0% discount.

----- Receipt -----
Burger (Main Course): $10.99
Fries (Side): $4.99
Soda (Beverage): $2.49

Subtotal: $18.47
Discount (15.0%): -$2.77
Tax (8.5%): $1.33
Total: $17.03
```

Program 5:

Code:

```
1  from datetime import datetime, timedelta
2
3  class Vehicle:
4
5      def __init__(self, license_plate, model, rental_price_per_day):
6          self.license_plate = license_plate.strip()
7          self.model = model.strip()
8          self.rental_price_per_day = float(rental_price_per_day)
9          self.available = True
10         self.rental_start_date = None
11
12     def rent(self, start_date=None):
13         if not self.available:
14             raise ValueError(f"Vehicle {self.license_plate} is not available for rent.")
15         if start_date is None:
16             start_date = datetime.now()
17         self.available = False
18         self.rental_start_date = start_date
19         print(f"Rented {self.__class__.__name__} {self.model} ({self.license_plate}) on {start_date.date()}.")
20
21     def return_vehicle(self, end_date=None):
22         if self.available:
23             raise ValueError(f"Vehicle {self.license_plate} is not currently rented.")
24         if end_date is None:
25             end_date = datetime.now()
26         if end_date < self.rental_start_date:
27             raise ValueError("End date cannot be before start date.")
28
29         delta = end_date - self.rental_start_date
30         days = delta.days + (1 if delta.seconds > 0 else 0)
31         if days < 1:
32             days = 1
33
34         cost = self.calculate_cost(days)
35         self.available = True
36         self.rental_start_date = None
37         print(f"Returned {self.__class__.__name__} {self.model} ({self.license_plate}) on {end_date.date()}. Days: {days}, Cost: ${cost:.2f}")
38         return cost
```

```

40     def calculate_cost(self, days):
41         if days <= 0:
42             raise ValueError("Days must be positive.")
43         return round(self.rental_price_per_day * days, 2)
44
45     def __str__(self):
46         status = "Available" if self.available else "Rented"
47         return f"{self.__class__.__name__}: {self.model} ({self.license_plate}) - ${self.rental_price_per_day:.2f}/day - {status}"
48
49
50 class Car(Vehicle):
51
52     def __init__(self, license_plate, model, rental_price_per_day, seats):
53         super().__init__(license_plate, model, rental_price_per_day)
54         if not isinstance(seats, int) or seats < 2:
55             raise ValueError("Seats must be an integer >= 2.")
56         self.seats = seats
57
58     def __str__(self):
59         return super().__str__() + f" - Seats: {self.seats}"
60
61
62 class Motorcycle(Vehicle):
63
64     def __init__(self, license_plate, model, rental_price_per_day, engine_cc):
65         super().__init__(license_plate, model, rental_price_per_day)
66         self.engine_cc = engine_cc
67
68     def __str__(self):
69         return super().__str__() + f" - Engine: {self.engine_cc}cc"

```

```

72 class Truck(Vehicle):
73
74     def __init__(self, license_plate, model, rental_price_per_day, load_capacity):
75         super().__init__(license_plate, model, rental_price_per_day)
76         if not isinstance(load_capacity, (int, float)) or load_capacity <= 0:
77             raise ValueError("Load capacity must be positive.")
78         self.load_capacity = float(load_capacity)
79
80     def __str__(self):
81         return super().__str__() + f" - Capacity: {self.load_capacity} tons"
82
83
84 class RentalSystem:
85
86     def __init__(self):
87         self.vehicles = {}
88
89     def add_vehicle(self, vehicle):
90         if vehicle.license_plate in self.vehicles:
91             raise ValueError(f"Vehicle with license {vehicle.license_plate} already exists.")
92         self.vehicles[vehicle.license_plate] = vehicle
93         print(f"Added: {vehicle}")
94
95     def rent_vehicle(self, license_plate, start_date=None):
96         if license_plate not in self.vehicles:
97             raise KeyError(f"Vehicle {license_plate} not found.")
98         vehicle = self.vehicles[license_plate]
99         vehicle.rent(start_date)
100
101     def return_vehicle(self, license_plate, end_date=None):
102         if license_plate not in self.vehicles:
103             raise KeyError(f"Vehicle {license_plate} not found.")
104         vehicle = self.vehicles[license_plate]
105         return vehicle.return_vehicle(end_date)

```

```

1107     def list_available_vehicles(self):
1108         available = [v for v in self.vehicles.values() if v.available]
1109         if not available:
1110             print("No vehicles available.")
1111         else:
1112             print("Available vehicles:")
1113             for v in available:
1114                 print(f" - {v}")
1115         return available
1116
1117 if __name__ == "__main__":
1118     system = RentalSystem()
1119
1120     car = Car("ABP123", "Toyota Camry", 50.0, 5)
1121     motorcycle = Motorcycle("BAP6349", "Honda CD70", 30.0, 600)
1122     truck = Truck("PAL325", "Ford F150", 80.0, 2.5)
1123
1124     system.add_vehicle(car)
1125     system.add_vehicle(motorcycle)
1126     system.add_vehicle(truck)
1127
1128     # Rent car on a specific date
1129     start_date = datetime(2025, 10, 14)
1130     system.rent_vehicle("ABP123", start_date)
1131
1132     # List available
1133     system.list_available_vehicles()
1134
1135     # Return after 3 days
1136     end_date = start_date + timedelta(days=3)
1137     cost = system.return_vehicle("ABP123", end_date)
1138     print(f"Total cost: ${cost:.2f}")
1139
1140     system.list_available_vehicles()

```

Output:

```

Added: Car: Toyota Camry (ABP123) - $50.00/day - Available - Seats: 5
Added: Motorcycle: Honda CD70 (BAP6349) - $30.00/day - Available - Engine: 600cc
Added: Truck: Ford F150 (PAL325) - $80.00/day - Available - Capacity: 2.5 tons
Rented Car Toyota Camry (ABP123) on 2025-10-14.
Available vehicles:
 - Motorcycle: Honda CD70 (BAP6349) - $30.00/day - Available - Engine: 600cc
 - Truck: Ford F150 (PAL325) - $80.00/day - Available - Capacity: 2.5 tons
Returned Car Toyota Camry (ABP123) on 2025-10-17. Days: 3, Cost: $150.00
Total cost: $150.00
Available vehicles:
 - Car: Toyota Camry (ABP123) - $50.00/day - Available - Seats: 5
 - Motorcycle: Honda CD70 (BAP6349) - $30.00/day - Available - Engine: 600cc
 - Truck: Ford F150 (PAL325) - $80.00/day - Available - Capacity: 2.5 tons

```

Program 6:

Code:

```
1  from datetime import datetime
2
3  class Employee:
4
5      def __init__(self, employee_id, name, position, salary, hire_date=None):
6          self.employee_id = employee_id
7          self.name = name
8          self.position = position
9          self.salary = float(salary)
10         self.hire_date = hire_date
11
12     def calculate_monthly_salary(self):
13         monthly = self.salary / 12
14         return round(monthly, 2)
15
16     def calculate_annual_salary(self):
17         return round(self.salary, 2)
18
19     def apply_raise(self, percentage):
20         if not isinstance(percentage, (int, float)) or percentage <= 0:
21             raise ValueError("Raise percentage must be positive.")
22         increase_factor = 1 + (percentage / 100)
23         old_salary = self.salary
24         self.salary *= increase_factor
25         self.salary = round(self.salary, 2)
26         print(f"Applied {percentage}% raise to {self.name}. New annual salary: ${self.salary:.2f} (from ${old_salary:.2f}).")
27
28     def get_employment_duration(self, as_of_date=None):
29         if as_of_date is None:
30             as_of_date = datetime.now()
31
32         delta = as_of_date - self.hire_date
33         days = delta.days
34         years = days / 365.25
35         return round(years, 2)
```

```

37     def display_details(self):
38         monthly = self.calculate_monthly_salary()
39         annual = self.calculate_annual_salary()
40         duration = self.get_employment_duration()
41
42         print(f"Employee ID: {self.employee_id}")
43         print(f"Name: {self.name}")
44         print(f"Position: {self.position}")
45         print(f"Hire Date: {self.hire_date.date()}")
46         print(f"Employment Duration: {duration} years")
47         print(f"Annual Salary: ${annual:.2f}")
48         print(f"Monthly Salary: ${monthly:.2f}")
49
50     if __name__ == "__main__":
51         # Create employee hired on a specific date
52         hire_date = datetime(2023, 1, 15)
53         emp = Employee("019", "Tayyab Imran", "Software Engineer", 80000, hire_date)
54
55         emp.display_details()
56
57         # Apply raise
58         emp.apply_raise(10)
59
60         # Update display
61         emp.display_details()
62
63         # Manual duration check
64         print(f"Duration as of now: {emp.get_employment_duration()} years")

```

Output:

```

Employee ID: 019
Name: Tayyab Imran
Position: Software Engineer
Hire Date: 2023-01-15
Employment Duration: 2.75 years
Annual Salary: $80000.00
Monthly Salary: $6666.67
Applied 10% raise to Tayyab Imran. New annual salary: $88000.00 (from $80000.00).
Employee ID: 019
Name: Tayyab Imran
Position: Software Engineer
Hire Date: 2023-01-15
Employment Duration: 2.75 years
Annual Salary: $88000.00
Monthly Salary: $7333.33
Duration as of now: 2.75 years

```

Program 7:

Code:

```
1 class Product:
2
3     def __init__(self, product_id, name, price, stock_quantity):
4         self.product_id = product_id.strip()
5         self.name = name.strip()
6         self.price = float(price)
7         self.stock_quantity = stock_quantity
8
9     def update_stock(self, quantity_change):
10        new_stock = self.stock_quantity + quantity_change
11        if new_stock < 0:
12            raise ValueError(f"Insufficient stock for {self.name}: only {self.stock_quantity} available.")
13        self.stock_quantity = new_stock
14
15    def __str__(self):
16        return f"{self.name} (ID: {self.product_id}) - ${self.price:.2f} - Stock: {self.stock_quantity}"
17
18    def __eq__(self, other):
19        if isinstance(other, Product):
20            return self.product_id == other.product_id
21        return False
22
23    def __hash__(self):
24        return hash(self.product_id)
```

```
27 class ShoppingCart:
28     def __init__(self):
29         self.items = {}
30
31     def add_product(self, product, quantity=1):
32         if product.stock_quantity < quantity:
33             raise ValueError(f"Insufficient stock for {product.name}: only {product.stock_quantity} available.")
34
35         if product in self.items:
36             new_qty = self.items[product] + quantity
37             if product.stock_quantity < new_qty:
38                 raise ValueError(f"Adding {quantity} would exceed available stock for {product.name}.")
39             self.items[product] = new_qty
40         else:
41             self.items[product] = quantity
42         print(f"Added {quantity} of {product.name} to cart.")
43
44     def remove_product(self, product, quantity=None):
45         if product not in self.items:
46             raise KeyError(f"{product.name} not in cart.")
47
48         if quantity is None:
49             del self.items[product]
50             print(f"Removed all {product.name} from cart.")
51         else:
52             if quantity > self.items[product]:
53                 raise ValueError(f"Cannot remove {quantity}; only {self.items[product]} in cart.")
54             self.items[product] -= quantity
55             if self.items[product] == 0:
56                 del self.items[product]
57             print(f"Removed {quantity} of {product.name} from cart.")
58
59     def calculate_total_cost(self):
60         total = sum(product.price * qty for product, qty in self.items.items())
61         return round(total, 2)
```

```

59     def calculate_total_cost(self):
60         total = sum(product.price * qty for product, qty in self.items.items())
61         return round(total, 2)
62
63     def view_cart(self):
64         if not self.items:
65             print("Cart is empty.")
66             return
67
68         print("Cart Contents:")
69         for product, qty in self.items.items():
70             subtotal = product.price * qty
71             print(f" - {product.name} x{qty} = ${subtotal:.2f}")
72         print(f"Total Cost: ${self.calculate_total_cost():.2f}")
73
74     def checkout(self):
75         if not self.items:
76             raise ValueError("Cannot checkout; cart is empty.")
77
78         total = self.calculate_total_cost()
79
80         for product, qty in self.items.items():
81             product.update_stock(-qty)
82
83         print(f"Checkout successful! Total charged: ${total:.2f}")
84         print("Inventory updated.")
85
86         self.items.clear()
87
88         return total

```

```

90 if __name__ == "__main__":
91     # Create products
92     p1 = Product("P001", "Laptop", 999.99, 10)
93     p2 = Product("P002", "Mouse", 25.50, 50)
94     p3 = Product("P003", "Keyboard", 75.00, 30)
95
96     # Display initial stock
97     print(p1)
98     print(p2)
99
100    # Create cart
101    cart = ShoppingCart()
102
103    # Add items
104    cart.add_product(p1, 2)
105    cart.add_product(p2, 3)
106
107    # View cart
108    cart.view_cart()
109
110    # Remove some
111    cart.remove_product(p2, 1)
112    cart.view_cart()
113
114    # Checkout
115    total = cart.checkout()
116
117    # Check updated stock
118    print(f"Updated stock for {p1.name}: {p1.stock_quantity}")
119    print(f"Updated stock for {p2.name}: {p2.stock_quantity}")

```

Output:

```
Laptop (ID: P001) - $999.99 - Stock: 10
Mouse (ID: P002) - $25.50 - Stock: 50
Added 2 of Laptop to cart.
Added 3 of Mouse to cart.
Cart Contents:
- Laptop x2 = $1999.98
- Mouse x3 = $76.50
Total Cost: $2076.48
Removed 1 of Mouse from cart.
Cart Contents:
- Laptop x2 = $1999.98
- Mouse x2 = $51.00
Total Cost: $2050.98
Checkout successful! Total charged: $2050.98
Inventory updated.
Updated stock for Laptop: 8
Updated stock for Mouse: 48
```


Program 8:

Code:

```
1  from datetime import datetime
2
3  class Patient:
4
5      def __init__(self, patient_id, name, age, medical_history=None):
6          self.patient_id = patient_id.strip()
7          self.name = name.strip()
8          self.age = age
9          self.medical_history = medical_history[:] if medical_history else [] # Copy list
10
11     def add_medical_history(self, entry):
12         if not entry or not entry.strip():
13             raise ValueError("Medical history entry cannot be empty.")
14         self.medical_history.append(entry.strip())
15         print(f"Added to {self.name}'s medical history: {entry.strip()}")
16
17     def view_medical_history(self):
18         print(f"Medical History for {self.name} (ID: {self.patient_id}):")
19         if not self.medical_history:
20             print("  No records.")
21         else:
22             for i, entry in enumerate(self.medical_history, 1):
23                 print(f"  {i}. {entry}")
24
25     def __str__(self):
26         return f"Patient: {self.name} (ID: {self.patient_id}, Age: {self.age})"
27
28     def __eq__(self, other):
29         if isinstance(other, Patient):
30             return self.patient_id == other.patient_id
31         return False
32
33     def __hash__(self):
34         return hash(self.patient_id)
```

```

37 class Appointment:
38
39     def __init__(self, appointment_id, patient, doctor_name, date_time):
40         self.appointment_id = appointment_id.strip()
41         self.patient = patient
42         self.doctor_name = doctor_name.strip()
43         self.date_time = date_time
44         self.status = 'Scheduled'
45
46     def cancel(self):
47         if self.status == 'Cancelled':
48             raise ValueError("Appointment is already cancelled.")
49         if self.status == 'Completed':
50             raise ValueError("Cannot cancel a completed appointment.")
51         self.status = 'Cancelled'
52         print(f"Appointment {self.appointment_id} cancelled.")
53
54     def complete(self):
55         if self.status == 'Cancelled':
56             raise ValueError("Cannot complete a cancelled appointment.")
57         self.status = 'Completed'
58         print(f"Appointment {self.appointment_id} marked as completed.")
59
60     def __str__(self):
61         return f"Appointment {self.appointment_id}: {self.patient.name} with Dr. {self.doctor_name} on {self.date_time.strftime('%Y-%m-%d %H:%M')}"

```

```

64 class AppointmentSystem:
65
66     def __init__(self):
67         self.appointments = {}
68
69     def schedule_appointment(self, appointment):
70         if appointment.appointment_id in self.appointments:
71             raise ValueError(f"Appointment ID {appointment.appointment_id} already exists.")
72         self.appointments[appointment.appointment_id] = appointment
73         print(f"Scheduled: {appointment}")
74
75     def cancel_appointment(self, appointment_id):
76         if appointment_id not in self.appointments:
77             raise KeyError(f"Appointment {appointment_id} not found.")
78         self.appointments[appointment_id].cancel()
79
80     def view_appointment(self, appointment_id):
81         if appointment_id not in self.appointments:
82             raise KeyError(f"Appointment {appointment_id} not found.")
83         appt = self.appointments[appointment_id]
84         print(appt)
85         return appt
86
87     def list_all_appointments(self, status_filter=None):
88         filtered = [appt for appt in self.appointments.values() if status_filter is None or appt.status == status_filter]
89         if not filtered:
90             print(f"No appointments found{' for status ' + status_filter if status_filter else ''}.".")
91         else:
92             print(f"Appointments{' (' + status_filter + ')' if status_filter else ''}:")
93             for appt in filtered:
94                 print(f"  - {appt}")
95         return filtered

```

```

97     def list_patient_appointments(self, patient_id):
98         patient_appts = [appt for appt in self.appointments.values() if appt.patient.patient_id == patient_id]
99         if not patient_appts:
100             print(f"No appointments for patient {patient_id}.")
101         else:
102             print(f"Appointments for patient {patient_id}.")
103             for appt in patient_appts:
104                 print(f" - {appt}")
105         return patient_appts
106
107
108 # Example usage:
109 if __name__ == "__main__":
110     # Create patient
111     patient = Patient("PAT001", "John Smith", 45)
112     patient.add_medical_history("Hypertension diagnosed in 2020")
113     patient.add_medical_history("Allergic to penicillin")
114     patient.view_medical_history()
115     # Create appointment system
116     system = AppointmentSystem()
117     # Schedule appointments
118     appt_date1 = datetime(2025, 10, 20, 10, 0) # Future date
119     appt1 = Appointment("AP001", patient, "Dr. Emily Brown", appt_date1)
120     system.schedule_appointment(appt1)
121     appt_date2 = datetime(2025, 11, 5, 14, 30)
122     appt2 = Appointment("AP002", patient, "Dr. Michael Lee", appt_date2)
123     system.schedule_appointment(appt2)
124     # View all
125     system.list_all_appointments()
126     # Cancel one
127     system.cancel_appointment("AP001")
128     # List patient's
129     system.list_patient_appointments("PAT001")
130     # View specific
131     system.view_appointment("AP002")

```

Output:

```

Added to John Smith's medical history: Hypertension diagnosed in 2020
Added to John Smith's medical history: Allergic to penicillin
Medical History for John Smith (ID: PAT001):
  1. Hypertension diagnosed in 2020
  2. Allergic to penicillin
Scheduled: Appointment AP001: John Smith with Dr. Dr. Emily Brown on 2025-10-20 10:00 - Status: Scheduled
Scheduled: Appointment AP002: John Smith with Dr. Dr. Michael Lee on 2025-11-05 14:30 - Status: Scheduled
Appointments:
  - Appointment AP001: John Smith with Dr. Dr. Emily Brown on 2025-10-20 10:00 - Status: Scheduled
  - Appointment AP002: John Smith with Dr. Dr. Michael Lee on 2025-11-05 14:30 - Status: Scheduled
Appointment AP001 cancelled.
Appointments for patient PAT001:
  - Appointment AP001: John Smith with Dr. Dr. Emily Brown on 2025-10-20 10:00 - Status: Cancelled
  - Appointment AP002: John Smith with Dr. Dr. Michael Lee on 2025-11-05 14:30 - Status: Scheduled
Appointment AP002: John Smith with Dr. Dr. Michael Lee on 2025-11-05 14:30 - Status: Scheduled

```

Program 9:

Code:

```
1  from datetime import datetime
2
3  class Post:
4      def __init__(self, content, privacy='public'):
5          self.content = content.strip()
6          self.timestamp = datetime.now()
7          self.privacy = privacy
8
9      def __str__(self):
10         return f"[{self.timestamp.strftime('%Y-%m-%d %H:%M:%S')}] ({self.privacy}): {self.content}"
11
12
13  class User:
14
15      def __init__(self, username, email):
16          if not username or not username.strip():
17              raise ValueError("Username cannot be empty.")
18          if not email or not email.strip():
19              raise ValueError("Email cannot be empty.")
20
21          self.username = username.strip()
22          self.email = email.strip()
23          self.friends = []
24          self.posts = []
25
26      def add_friend(self, other_user):
27          if other_user in self.friends:
28              raise ValueError(f"{other_user.username} is already a friend.")
29
30          self.friends.append(other_user)
31          if self not in other_user.friends:
32              other_user.friends.append(self)
33          print(f"{self.username} and {other_user.username} are now friends.")
34
35      def create_post(self, content, privacy='public'):
36          post = Post(content, privacy)
37          self.posts.append(post)
38          print(f"{self.username} created a new post: {post}")
39          return post
```

```

41     def display_timeline(self):
42         visible_posts = []
43
44         for post in self.posts:
45             visible_posts.append((post.timestamp, self.username, post))
46
47         for friend in self.friends:
48             for post in friend.posts:
49                 if post.privacy in ['public', 'friends']:
50                     visible_posts.append((post.timestamp, friend.username, post))
51
52         visible_posts.sort(key=lambda x: x[0], reverse=True)
53
54         print(f"Timeline for {self.username}:")
55         if not visible_posts:
56             print("  No posts to display.")
57         else:
58             for ts, author, post in visible_posts:
59                 print(f"    {author}: {post}")
60
61 if __name__ == "__main__":
62     user1 = User("Tayyab", "tjtayyab28@gmail.com")
63     user2 = User("Zain", "mianzain@gmail.com")
64     user3 = User("Haseeb", "haseeb@gmail.com")
65
66     user1.add_friend(user2)
67     user1.add_friend(user3)
68
69     user1.create_post("Hello world!", "public")
70     user2.create_post("Private thought.", "private")
71     user2.create_post("Sharing with friends.", "friends")
72     user3.create_post("Public announcement.", "public")
73
74     user1.display_timeline()
75
76     user2.display_timeline()

```

Output:

```

Tayyab and Zain are now friends.
Tayyab and Haseeb are now friends.
Tayyab created a new post: [2025-10-14 16:10:29] (public): Hello world!
Zain created a new post: [2025-10-14 16:10:29] (private): Private thought.
Zain created a new post: [2025-10-14 16:10:29] (friends): Sharing with friends.
Haseeb created a new post: [2025-10-14 16:10:29] (public): Public announcement.
Timeline for Tayyab:
  Haseeb: [2025-10-14 16:10:29] (public): Public announcement.
  Zain: [2025-10-14 16:10:29] (friends): Sharing with friends.
  Tayyab: [2025-10-14 16:10:29] (public): Hello world!
Timeline for Zain:
  Zain: [2025-10-14 16:10:29] (friends): Sharing with friends.
  Zain: [2025-10-14 16:10:29] (private): Private thought.
  Tayyab: [2025-10-14 16:10:29] (public): Hello world!

```

Program 10:

Code:

```
1  from datetime import datetime, timedelta
2
3  class Room:
4
5      def __init__(self, room_number, room_type, price_per_night):
6          self.room_number = room_number.strip()
7          self.room_type = room_type
8          self.price_per_night = float(price_per_night)
9          self.available = True
10
11     def book(self):
12         if not self.available:
13             raise ValueError(f"Room {self.room_number} is already booked.")
14         self.available = False
15         print(f"Room {self.room_number} booked.")
16
17     def release(self):
18         if self.available:
19             raise ValueError(f"Room {self.room_number} is already available.")
20         self.available = True
21         print(f"Room {self.room_number} released.")
22
23     def __str__(self):
24         status = "Available" if self.available else "Booked"
25         return f"Room {self.room_number} ({self.room_type}) - ${self.price_per_night:.2f}/night - {status}"
26
27
28 class Booking:
29
30     def __init__(self, booking_id, room, guest_name, check_in, check_out):
31         self.booking_id = booking_id.strip()
32         self.room = room
33         self.guest_name = guest_name.strip()
34         self.check_in = check_in
35         self.check_out = check_out
36
37         self.room.book()
38
39         self.total_cost = self.calculate_total_cost()
40         print(f"Booking {self.booking_id} created for {self.guest_name} in {self.room}.")
41
42     def calculate_total_cost(self):
43         delta = self.check_out - self.check_in
44         nights = delta.days
45         if delta.seconds > 0:
46             nights += 1
47         if nights < 1:
48             nights = 1
49
50         cost = self.room.price_per_night * nights
51         return round(cost, 2)
52
53     def checkout(self):
54         if self.room.available:
55             raise ValueError(f"Room {self.room.room_number} is already available (checkout done).")
56         self.room.release()
57         print(f"Checkout for booking {self.booking_id}. Total cost: ${self.total_cost:.2f}")
58         return self.total_cost
59
60     def __str__(self):
61         return f"Booking {self.booking_id}: {self.guest_name} in Room {self.room.room_number} from {self.check_in.strftime('%Y-%m-%d')}
```

```

64 class HotelSystem:
65
66     def __init__(self):
67         self.rooms = {}
68         self.bookings = {}
69
70     def add_room(self, room):
71         if room.room_number in self.rooms:
72             raise ValueError(f"Room {room.room_number} already exists.")
73         self.rooms[room.room_number] = room
74         print(f"Added: {room}")
75
76     def create_booking(self, booking):
77         if booking.booking_id in self.bookings:
78             raise ValueError(f"Booking {booking.booking_id} already exists.")
79         self.bookings[booking.booking_id] = booking
80         print(f"Booking recorded: {booking}")
81
82     def checkout_booking(self, booking_id):
83         if booking_id not in self.bookings:
84             raise KeyError(f"Booking {booking_id} not found.")
85         booking = self.bookings[booking_id]
86         cost = booking.checkout()
87         return cost
88
89     def list_available_rooms(self):
90         available = [r for r in self.rooms.values() if r.available]
91         if not available:
92             print("No rooms available.")
93         else:
94             print("Available rooms:")
95             for r in available:
96                 print(f" - {r}")
97         return available

```

```

99 if __name__ == "__main__":
100     # Create rooms
101     room1 = Room("101", "single", 80.0)
102     room2 = Room("202", "double", 120.0)
103     room3 = Room("303", "suite", 250.0)
104
105     # Hotel system
106     hotel = HotelSystem()
107     hotel.add_room(room1)
108     hotel.add_room(room2)
109     hotel.add_room(room3)
110
111     # Create booking
112     check_in = datetime(2025, 10, 20, 15, 0)
113     check_out = check_in + timedelta(days=3)
114     booking1 = Booking("B001", room1, "John Doe", check_in, check_out)
115     hotel.create_booking(booking1)
116
117     # List available
118     hotel.list_available_rooms()
119
120     # Checkout
121     hotel.checkout_booking("B001")
122
123     # List available again
124     hotel.list_available_rooms()

```

Output:

```
Added: Room 101 (single) - $80.00/night - Available
Added: Room 202 (double) - $120.00/night - Available
Added: Room 303 (suite) - $250.00/night - Available
Room 101 booked.
Booking B001 created for John Doe in Room 101 (single) - $80.00/night - Booked.
Booking recorded: Booking B001: John Doe in Room 101 from 2025-10-20 to 2025-10-23 - Total: $240.00
Available rooms:
- Room 202 (double) - $120.00/night - Available
- Room 303 (suite) - $250.00/night - Available
Room 101 released.
Checkout for booking B001. Total cost: $240.00
Available rooms:
- Room 101 (single) - $80.00/night - Available
- Room 202 (double) - $120.00/night - Available
- Room 303 (suite) - $250.00/night - Available
```